

省博物馆参观预约系统 · 系统设计文档

省博物馆参观预约系统系统设计文档（终稿）

本文档为系统设计终稿，内容已与实际落地的代码、数据库 schema.sql、后端接口、前端页面以及部署脚本对齐，可直接作为大作业「系统设计文档」提交（导出为 PDF）。

1. 文档说明

1.1 文档目的

本文档是 省博物馆参观预约系统 的系统设计终稿，覆盖需求描述、功能模块、系统架构、数据库设计、核心业务流程、并发处理方案以及三种并发方案对比与选型理由，用于指导开发并作为最终提交材料。

本文档根据以下文件整理生成：

- 项目要求分析.md
- 项目设计开发流程.md
- 需求拆分.md
- 大作业共性要求.docx
- web大作业-02-省博物馆参观预约系统.docx

1.2 项目当前说明

本系统已按本文档完成开发并通过功能测试与并发压测，技术栈为 Vue 3 + Axios + Bootstrap 前端、Spring Boot 后端、MySQL 8 数据库，并提供 Linux/VMware 一键部署脚本。系统已落地以下内容：

- 正式数据库表结构（见 museum-reservation-system/database/schema.sql）
- 正式后端接口（Spring Boot REST API）
- 正式业务规则（时间/名额/单人上限/防重复校验）
- 正式前端页面结构（游客端 + 管理员端）
- 管理员端和游客端模块划分
- 并发控制和数据一致性方案（数据库条件原子更新 + 唯一索引 + 事务）

2. 项目背景与总体目标

2.1 项目背景

博物馆在节假日、热门展览和旅游高峰期间，容易出现游客集中到馆、排队拥堵、接待压力过大等问题。为了控制每日参观人数、优化入馆秩序、提升游客体验，需要建设一个支持 **实名预约**、**限量预约**、**分时段预约**、**后台配置**、**预约统计**和**并发控制**的参观预约系统。

2.2 总体目标

本系统需要实现一个完整可运行、可部署、可演示的省博物馆参观预约系统。

系统目标如下：

游客可以实名登录，查看场馆信息，选择日期和时段进行预约，并查看预约凭证；
管理员可以维护场馆信息、通知公告、预约活动、预约时段和名额；
系统需要在高并发预约场景下保证不超额、不重复、名额数据准确一致；

系统最终需要部署到 VMware Linux 虚拟机，并保证宿主机浏览器可以访问。

3. 需求描述

3.1 用户角色

系统包含两个主要角色：

- **游客用户**：使用系统进行参观预约。
- **管理员用户**：使用后台管理系统维护预约规则和查看预约数据。

3.2 游客端需求

游客端需要完成从登录到预约再到查看凭证的完整流程。

主要功能包括：

- **实名登录**：游客使用身份证号、手机号和姓名进行登录。
- **场馆信息浏览**：查看博物馆开放时间、地址、参观须知和预约规则。
- **通知公告浏览**：查看特殊展览、临时闭馆、预约提醒等通知。
- **可预约时段查询**：查看可预约日期、入场时段、总名额、已预约人数和剩余名额。
- **提交预约**：选择日期和时段后提交预约请求。
- **预约结果查看**：查看预约成功或失败原因。
- **我的预约记录**：查看本人历史预约记录。
- **预约凭证和二维码**：预约成功后生成预约编号和二维码凭证。

游客预约结果至少需要覆盖：

- 预约成功
- 名额已满
- 已超过预约上限
- 未到预约时间
- 预约已结束
- 预约通道已关闭
- 重复预约失败
- 身份证号或手机号格式错误

3.3 管理员端需求

管理员端用于维护系统基础信息、预约规则和预约数据。

主要功能包括：

- **管理员登录**：管理员使用用户名和密码登录后台。
- **场馆信息管理**：维护场馆名称、地址、开放时间、参观须知和预约规则。
- **通知公告管理**：发布、编辑、启用、禁用公告通知。
- **预约活动配置**：配置参观日期、每日总名额、预约开始时间、预约结束时间、单人预约上限和活动状态。
- **分时段名额管理**：配置每个预约活动下的入场时段、时段总名额和启用状态。
- **预约通道控制**：支持启用或禁用预约活动、预约时段。
- **预约记录管理**：查看成功预约记录和游客信息。
- **预约名单导出**：支持导出预约名单，优先采用 CSV 格式。
- **预约统计查看**：查看游客数、预约总数、总名额、已预约名额、剩余名额和分时段统计。
- **系统状态查看**：查看预约活动状态、预约接口请求量和系统健康状态。

3.4 非功能需求

3.4.1 高并发与稳定性

系统需要支持大量游客同时预约，尤其是节假日和热门展览场景。

要求：

- 预约接口高并发下不崩溃
- 接口能够正常返回成功或失败结果
- 页面不长时间卡死
- 数据库不因并发请求产生错误数据

3.4.2 数据一致性

数据一致性是本项目最重要的评分点之一。

必须保证：

- 不能超额预约
- 不能重复预约
- 预约记录数与已预约名额保持一致
- 多人同时抢同一时段名额时，成功人数不能超过配置名额

3.4.3 时间严格控制

系统必须严格控制预约开放时间。

要求：

- 未到预约开放时间不能预约
- 超过预约结束时间不能预约
- 预约活动结束后不能继续预约
- 预约通道关闭后不能预约

3.4.4 可部署性

系统必须部署到指定环境。

要求：

- VMware 虚拟机
- Linux 系统，Ubuntu 或 CentOS
- 固定 8GB 内存，即 8192MB
- 虚拟机中安装 JDK 和 MySQL
- 项目可在虚拟机中打包部署运行
- 宿主机浏览器可以访问系统功能

3.4.5 文档与交付要求

最终提交材料需要包含：

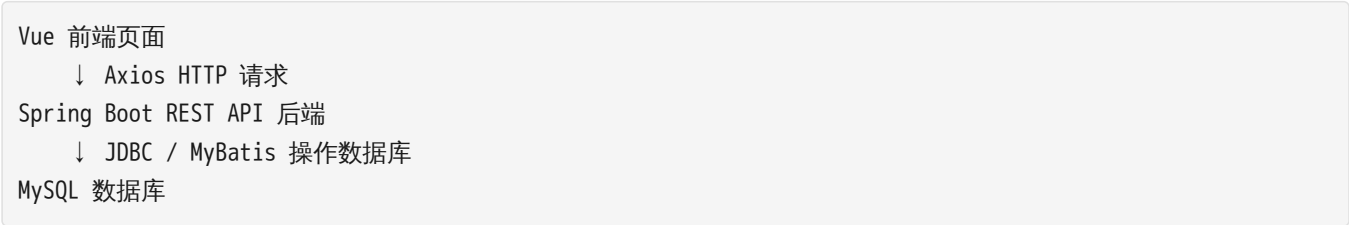
- 系统设计文档 PDF
 - 功能测试报告 PDF
 - 性能测试报告 PDF
 - 系统演示视频
 - 完整源代码工程文件
-

4. 系统总体架构

4.1 架构模式

系统采用 前后端分离架构。

整体调用关系如下：

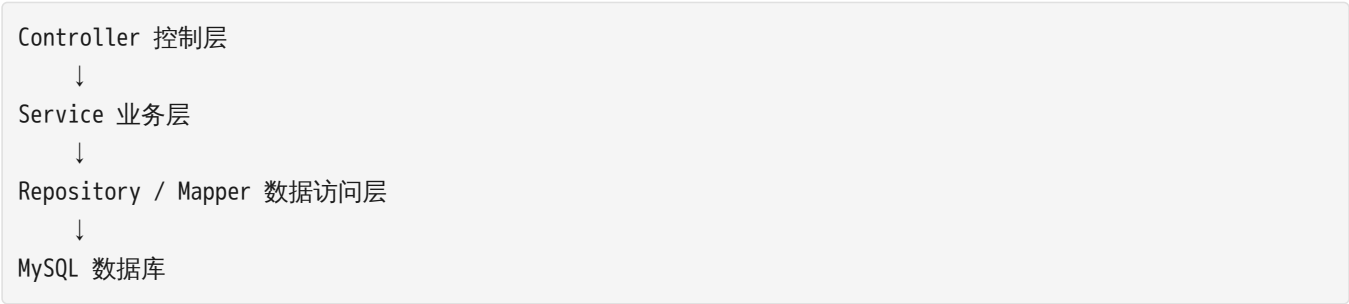


4.2 技术选型

层级	技术	作用
前端	Vue	构建用户端和管理员端页面
前端	Axios	向后端 REST API 发送 HTTP 请求
前端	Bootstrap	快速实现页面布局和基础样式
后端	Spring Boot	提供 REST API、业务逻辑和事务控制
数据库	MySQL	存储用户、管理员、场馆、公告、预约活动、时段和预约记录
部署	VMware + Linux	满足作业虚拟机部署要求
运行环境	JDK	运行 Spring Boot 后端服务

4.3 分层设计

后端采用典型分层结构：



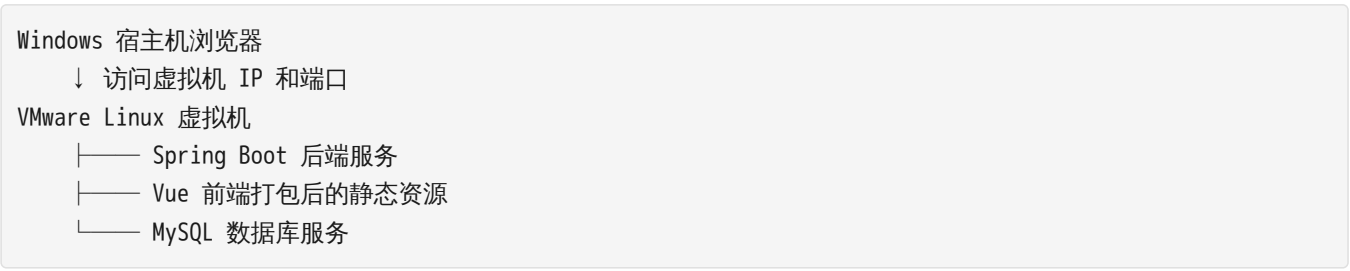
各层职责如下：

- **Controller 控制层**：接收前端请求，调用业务服务，返回统一响应。
- **Service 业务层**：处理登录、预约、名额扣减、重复预约校验、统计等核心逻辑。
- **Repository / Mapper 数据访问层**：负责数据库增删改查。
- **DTO 数据传输对象**：封装前端请求参数和后端响应数据。
- **Config 配置层**：处理跨域、统一异常、日期格式等配置。

4.4 部署架构

正式部署时，系统部署在 VMware Linux 虚拟机中。

部署关系如下：



大作业阶段推荐部署方式：

Vue 前端执行 npm build 后，将 dist 静态资源集成到 Spring Boot 静态目录，最终在虚拟机中启动一个 Spring Boot jar 包。

该方式部署简单，演示时只需要启动一个后端服务即可访问完整系统。

5. 功能模块设计

5.1 模块总览

系统功能模块划分如下：



5.2 游客端模块设计

5.2.1 游客实名登录模块

游客输入姓名、身份证号和手机号后登录系统。

设计要点：

- 校验身份证号格式
- 校验手机号格式
- 游客不存在时自动创建游客信息
- 游客存在时更新必要信息
- 登录后前端保存游客基本信息用于后续查询预约记录

5.2.2 场馆信息浏览模块

游客可以查看博物馆开放时间、地址、参观须知和预约规则。

设计要点：

- 数据由管理员后台维护
- 游客端只读展示
- 显示最近更新时间

5.2.3 通知公告浏览模块

游客可以查看已启用的公告通知。

设计要点：

- 只展示启用状态公告
- 按发布时间倒序排列
- 支持公告类型区分，例如普通通知、闭馆通知、特殊展览

5.2.4 可预约时段查询模块

游客查看可预约日期和时段。

设计要点：

- 显示参观日期、时段名称、入场时间、总名额、剩余名额
- 前端根据时段状态显示可预约、未开始、已结束、已满、已关闭等状态
- 前端仅做提示，真正校验必须在后端完成

5.2.5 提交预约模块

游客选择时段后提交预约。

设计要点：

- 预约接口必须放在事务中执行
- 必须防止超额预约
- 必须防止重复预约
- 必须校验预约时间范围
- 必须返回明确失败原因

5.2.6 我的预约记录模块

游客根据身份证号和手机号查询自己的预约记录。

设计要点：

- 只能查询本人记录
- 按预约时间倒序排列
- 显示预约编号、参观日期、时段、状态和二维码内容

5.2.7 预约凭证与二维码模块

预约成功后生成预约编号和二维码内容。

设计要点：

二维码内容 = MUSEUM_RESERVATION + 预约编号

大作业阶段只需要展示二维码凭证，不强制实现扫码核销。

5.3 管理员端模块设计

5.3.1 管理员登录模块

管理员使用用户名和密码登录后台。

设计要点：

- 管理员账号可通过初始化数据预置
- 大作业阶段使用简单 token 或登录状态即可
- 暂不引入复杂 Spring Security，降低配置难度

5.3.2 场馆信息管理模块

管理员维护场馆基础信息。

设计要点：

- 维护场馆名称、地址、开放时间、参观须知、预约规则 and 状态
- 游客端读取最新场馆信息
- 修改后更新 updated_at

5.3.3 通知公告管理模块

管理员管理公告通知。

设计要点：

- 支持新增、编辑、启用、禁用、删除公告
- 公告可区分普通通知、特殊展览、临时闭馆等类型

5.3.4 预约活动配置模块

管理员配置预约活动。

设计要点：

- 配置参观日期
- 配置每日总名额
- 配置预约开放时间和结束时间
- 配置单人预约上限
- 配置活动状态

5.3.5 分时时段名额管理模块

管理员配置具体入场时段。

设计要点：

- 每个活动下可以配置多个时段
- 每个时段有独立总名额和已预约人数
- 调整总名额时不能低于已预约人数
- 时段禁用后不能继续预约

5.3.6 预约记录管理模块

管理员查看预约记录。

设计要点：

- 支持按日期、时段、身份证号、手机号、状态筛选
- 显示预约编号、游客信息、参观日期、入场时段和预约时间

5.3.7 预约名单导出模块

管理员导出预约名单。

设计要点：

- 优先实现 CSV 导出
- 导出字段包含预约编号、游客姓名、身份证号、手机号、参观日期、入场时段和预约时间

5.3.8 预约统计模块

管理员查看系统统计数据。

设计要点：

- 游客总数
- 预约总数
- 总名额
- 已预约名额
- 剩余名额
- 各日期预约人数
- 各时段预约人数

5.3.9 系统状态模块

管理员查看系统状态。

设计要点：

- 系统健康状态
- 预约活动状态
- 预约接口累计请求数
- 最近请求时间
- 预约成功数和失败数

6. 数据库设计

6.1 数据库设计原则

数据库设计需要重点支持：

- 实名登录
- 后台配置
- 分时预约
- 名额控制
- 重复预约控制
- 预约记录查询
- 统计分析
- 并发数据一致性

6.2 数据表总览

正式系统建议设计以下数据表：

表名	中文名称	主要作用
admin_user	管理员表	保存管理员账号信息
visitor	游客表	保存游客实名信息
museum_info	场馆信息表	保存场馆开放信息和参观须知
notice	通知公告表	保存特殊展览、临时闭馆等公告
reservation_activity	预约活动表	保存某天或某阶段的预约规则
reservation_slot	预约时段表	保存分时时段和名额信息
reservation_record	预约记录表	保存游客预约成功记录
request_log	请求日志表，可选	保存预约请求量、成功失败统计或简单系统状态

6.3 管理员表 admin_user

用于管理员后台登录。

字段名	类型建议	说明
id	BIGINT	主键
username	VARCHAR(50)	管理员用户名，唯一
password	VARCHAR(100)	管理员密码，初期可明文，后续可加密
role	VARCHAR(30)	角色，例如 ADMIN
status	TINYINT	状态，1 启用，0 禁用
created_at	DATETIME	创建时间
updated_at	DATETIME	更新时间

约束设计：

- username 唯一
- status 默认为 1

6.4 游客表 visitor

用于保存游客实名登录信息。

字段名	类型建议	说明
id	BIGINT	主键
name	VARCHAR(50)	游客姓名
id_card	VARCHAR(18)	身份证号
phone	VARCHAR(20)	手机号
created_at	DATETIME	创建时间
updated_at	DATETIME	更新时间

约束设计：

- id_card 唯一
- 手机号需要做格式校验

6.5 场馆信息表 museum_info

用于保存博物馆基本信息。

字段名	类型建议	说明
id	BIGINT	主键
name	VARCHAR(100)	场馆名称
address	VARCHAR(255)	场馆地址
open_info	TEXT	开放时间说明
rules	TEXT	参观须知和预约规则
status	TINYINT	场馆状态，1 正常，0 停用
updated_at	DATETIME	更新时间

设计说明：

- 系统至少保留一条场馆信息
- 游客端首页读取该表展示场馆信息

6.6 通知公告表 notice

用于保存通知公告。

字段名	类型建议	说明
id	BIGINT	主键
title	VARCHAR(100)	公告标题
content	TEXT	公告内容
type	VARCHAR(30)	公告类型

enabled	TINYINT	是否启用，1 启用，0 禁用
created_at	DATETIME	创建时间
updated_at	DATETIME	更新时间

设计说明：

- 游客端只展示启用公告
- 后台可以新增、编辑、启用、禁用和删除公告

6.7 预约活动表 reservation_activity

用于保存某一天或某阶段的预约规则。

字段名	类型建议	说明
id	BIGINT	主键
activity_name	VARCHAR(100)	活动名称
visit_date	DATE	参观日期
daily_capacity	INT	每日总名额
person_limit	INT	单人预约上限
booking_start	DATETIME	预约开始时间
booking_end	DATETIME	预约结束时间
status	VARCHAR(20)	状态：未开始、进行中、已结束、禁用
created_at	DATETIME	创建时间
updated_at	DATETIME	更新时间

约束设计：

- booking_end 必须晚于 booking_start
- daily_capacity 必须大于 0
- person_limit 必须大于 0
- 同一参观日期建议只配置一个有效活动

6.8 预约时段表 reservation_slot

用于保存每个预约活动下的分时时段和名额。

字段名	类型建议	说明
id	BIGINT	主键
activity_id	BIGINT	所属预约活动 ID
visit_date	DATE	参观日期
slot_name	VARCHAR(50)	时段名称，例如 09:00-11:00
start_time	TIME	入场开始时间
end_time	TIME	入场结束时间

total_capacity	INT	时段总名额
booked_count	INT	已预约人数
enabled	TINYINT	是否启用
version	INT	版本号，用于并发控制
created_at	DATETIME	创建时间
updated_at	DATETIME	更新时间

约束设计：

- booked_count 不能大于 total_capacity
- total_capacity 必须大于 0
- 调整总名额时不能低于已预约人数
- activity_id 关联 reservation_activity.id

剩余名额计算：

```
remaining = total_capacity - booked_count
```

6.9 预约记录表 reservation_record

用于保存游客预约成功后的记录。

字段名	类型建议	说明
id	BIGINT	主键
reservation_no	VARCHAR(50)	预约编号，唯一
visitor_id	BIGINT	游客 ID
activity_id	BIGINT	预约活动 ID
slot_id	BIGINT	预约时段 ID
id_card	VARCHAR(18)	身份证号，冗余保存便于查询
phone	VARCHAR(20)	手机号，冗余保存便于查询
visit_date	DATE	参观日期
slot_name	VARCHAR(50)	预约时段名称
status	VARCHAR(20)	预约状态，例如 SUCCESS、CANCELLED
qr_content	VARCHAR(255)	二维码内容
created_at	DATETIME	预约时间

关键约束：

```
reservation_no 唯一
id_card + visit_date 唯一，防止同一身份证同一天重复预约
```

如果后续允许同一身份证同一天预约多次，则可以调整为：

id_card + slot_id 唯一，防止同一身份证重复预约同一时段

本系统初期建议采用：

同一身份证同一天只能预约一次

6.10 请求日志表 request_log，可选

用于统计预约请求量和系统状态。

字段名	类型建议	说明
id	BIGINT	主键
request_type	VARCHAR(50)	请求类型，例如 RESERVATION
success	TINYINT	是否成功
message	VARCHAR(255)	返回信息
created_at	DATETIME	请求时间

设计说明：

- 可用于统计预约接口累计请求数
- 可用于系统状态页面展示成功数、失败数和最近请求时间
- 如果时间紧张，该表可以后续补充

7. 接口设计初稿

7.1 统一响应格式

后端接口统一返回如下结构：

```
{
  "success": true,
  "message": "操作成功",
  "data": {}
}
```

字段说明：

字段	说明
success	是否成功
message	返回提示信息
data	返回数据

7.2 游客端接口

接口	方法	说明
/api/visitor/login	POST	游客实名登录

/api/museum/info	GET	查询场馆信息
/api/notices	GET	查询已启用公告
/api/slots	GET	查询可预约时段
/api/reservations	POST	提交预约
/api/reservations/my	GET	查询我的预约记录

7.3 管理员端接口

接口	方法	说明
/api/admin/login	POST	管理员登录
/api/admin/museum/info	GET	查询场馆信息
/api/admin/museum/info	PUT	修改场馆信息
/api/admin/notices	GET	查询公告列表
/api/admin/notices	POST	新增公告
/api/admin/notices/{id}	PUT	修改公告
/api/admin/notices/{id}	DELETE	删除公告
/api/admin/activities	GET	查询预约活动
/api/admin/activities	POST	新增预约活动
/api/admin/activities/{id}	PUT	修改预约活动
/api/admin/slots	GET	查询时段名额
/api/admin/slots	POST	新增预约时段
/api/admin/slots/{id}	PUT	修改预约时段
/api/admin/reservations	GET	查询预约记录
/api/admin/reservations/export	GET	导出预约名单
/api/admin/summary	GET	后台统计
/api/admin/status	GET	系统状态查看

7.4 关键接口说明

7.4.1 提交预约接口

接口：

POST /api/reservations

请求参数：

参数	说明
idCard	身份证号

phone	手机号
slotId	预约时段 ID

返回结果：

- **预约成功：**返回预约编号、参观日期、时段、二维码内容。
- **预约失败：**返回失败原因，例如名额已满、重复预约、未到预约时间等。

7.4.2 查询可预约时段接口

接口：

```
GET /api/slots
```

返回字段：

- 参观日期
- 时段名称
- 入场开始时间
- 入场结束时间
- 总名额
- 已预约人数
- 剩余名额
- 预约开始时间
- 预约结束时间
- 是否启用

8. 核心业务流程设计

8.1 游客预约主流程

游客预约主流程如下：



数据库原子扣减名额



写入预约记录



返回预约编号和二维码凭证



游客查看我的预约记录

8.2 提交预约详细流程

提交预约是系统最核心的流程。

1. 前端提交 idCard、phone、slotId
2. 后端校验身份证号格式
3. 后端校验手机号格式
4. 查询预约时段是否存在
5. 查询预约活动是否有效
6. 判断预约通道是否启用
7. 判断当前时间是否在预约开放时间内
8. 判断是否重复预约
9. 判断是否超过单人预约上限
10. 使用数据库条件更新扣减名额
11. 如果扣减失败，返回名额已满或时段不可预约
12. 如果扣减成功，插入预约记录
13. 生成预约编号和二维码内容
14. 返回预约成功结果

8.3 管理员配置预约活动流程

管理员登录后台



进入预约活动配置页面



新增或编辑预约活动



设置参观日期、每日总名额、预约开始时间、预约结束时间、单人预约上限



保存预约活动



进入分时时段配置页面



新增上午、下午或具体时间段



设置每个时段的总名额和启用状态



保存配置



游客端可以查询并预约该活动时段

8.4 管理员查看预约记录流程

管理员登录后台
↓
进入预约记录管理页面
↓
选择日期、时段、身份证号、手机号或状态筛选
↓
系统查询预约记录
↓
页面展示预约编号、游客信息、参观日期、时段和预约时间
↓
管理员可以导出预约名单

8.5 并发预约数据一致性流程

多个游客同时提交同一时段预约请求
↓
后端每个请求进入事务
↓
数据库执行条件更新扣减名额
↓
只有 `booked_count < total_capacity` 的请求可以更新成功
↓
更新成功的请求写入预约记录
↓
更新失败的请求返回名额已满
↓
最终成功预约记录数不超过时段总名额

9. 并发处理方案设计

9.1 并发问题分析

预约系统最容易出现的问题是多个用户同时抢同一个时段名额。

如果使用错误方式：

先查询剩余名额
如果剩余名额 > 0
再普通更新 `booked_count`

在高并发情况下，多个请求可能同时读到相同剩余名额，从而导致超额预约。

例如：

某时段只剩 1 个名额
用户 A 和用户 B 同时查询，都看到剩余名额为 1
两个请求都继续执行预约
最终成功 2 人，产生超额预约

因此不能只依赖前端判断，也不能只用普通查询加普通更新。

9.2 推荐并发方案

本系统推荐采用：

数据库条件更新 + 唯一约束 + 事务控制

核心 SQL 思路如下：

```
UPDATE reservation_slot
SET booked_count = booked_count + 1,
    version = version + 1
WHERE id = ?
    AND enabled = 1
    AND booked_count < total_capacity;
```

如果影响行数为 1，说明名额扣减成功。

如果影响行数为 0，说明：

- 名额已满
- 时段已禁用
- 预约条件不满足

重复预约通过数据库唯一约束保证：

id_card + visit_date 唯一

事务控制保证：

名额扣减和预约记录写入要么同时成功，要么同时失败。

9.3 事务设计

提交预约接口需要使用事务。

事务内主要操作：

1. 校验请求参数
2. 查询预约时段和活动信息
3. 校验时间规则、活动状态和预约上限
4. 条件更新扣减名额
5. 插入预约记录
6. 返回预约结果

如果任一步骤失败，需要回滚事务，避免出现：

- 名额增加了但没有预约记录
- 预约记录存在但名额没有增加

9.4 唯一约束设计

为防止重复预约，建议在预约记录表中增加唯一约束：

id_card + visit_date

业务含义：

同一身份证同一天只能预约一次。

如果后续允许同一天预约多个时段，可调整为：

`id_card + slot_id`

9.5 名额一致性校验

压测完成后需要检查：

`reservation_slot.booked_count = reservation_record 中对应时段 SUCCESS 记录数`

并且：

`reservation_slot.booked_count <= reservation_slot.total_capacity`

10. 三种并发方案对比及选型理由

10.1 方案一：数据库悲观锁

10.1.1 方案说明

在事务中查询预约时段时使用数据库锁，例如 `SELECT ... FOR UPDATE`，锁定对应时段记录后再判断名额并更新。

流程：

```
graph TD
    A[开启事务] --> B[查询时段并加行锁]
    B --> C[判断剩余名额]
    C --> D[扣减名额]
    D --> E[插入预约记录]
    E --> F[提交事务]
```

10.1.2 优点

- 逻辑直观
- 数据安全性强
- 适合强一致性场景

10.1.3 缺点

- 高并发时锁等待较多
- 性能可能下降

- 如果事务执行时间过长，容易影响吞吐量

10.1.4 适用场景

适合并发量不大、但数据一致性要求非常强的业务。

10.2 方案二：数据库乐观锁

10.2.1 方案说明

在预约时段表中增加 `version` 字段，更新时带上当前版本号。如果版本号一致则更新成功，否则说明数据已被其他请求修改，需要重试或返回失败。

核心 SQL 思路：

```
UPDATE reservation_slot
SET booked_count = booked_count + 1,
    version = version + 1
WHERE id = ?
    AND version = ?
    AND booked_count < total_capacity;
```

10.2.2 优点

- 不需要长时间持有锁
- 适合读多写少场景
- 并发性能通常优于悲观锁

10.2.3 缺点

- 更新失败后可能需要重试
- 代码逻辑比悲观锁复杂
- 高并发抢购场景下失败率可能较高

10.2.4 适用场景

适合读多写少、冲突不太频繁的业务场景。

10.3 方案三：条件更新 + 唯一约束 + 事务

10.3.1 方案说明

通过数据库条件更新保证名额不超过总容量，通过唯一约束保证同一用户不重复预约，通过事务保证名额扣减和预约记录写入一致。

核心 SQL 思路：

```
UPDATE reservation_slot
SET booked_count = booked_count + 1,
    version = version + 1
WHERE id = ?
    AND enabled = 1
    AND booked_count < total_capacity;
```

同时在预约记录表中设置唯一约束（与本项目实际落地一致，按时段维度防重复）：

```
uk_reservation_record_id_card_slot (id_card, slot_id)
```

并辅以 CHECK 约束 `booked_count <= total_capacity` 作为数据库层兜底。

10.3.2 优点

- 实现相对简单
- 能有效防止超额预约
- 能有效防止重复预约
- 适合课程大作业实现、演示和压测
- 不需要引入 Redis、消息队列等额外组件

10.3.3 缺点

- 极高并发情况下仍依赖数据库性能
- 如果业务进一步复杂，可能需要更完整的限流或缓存方案

10.3.4 适用场景

适合本课程大作业中的分时预约系统，既能满足数据一致性要求，又不会引入过高实现复杂度。

10.4 最终选型理由

本项目最终选择：

```
方案三：条件更新 + 唯一约束 + 事务
```

选型理由：

- 能够满足作业最核心的无超额预约要求
- 能够通过数据库唯一约束防止重复预约
- 能够通过事务保证名额和记录一致
- 实现难度适合课程大作业
- 不依赖额外中间件，部署到 VMware Linux 更简单
- 便于压测和展示数据一致性结果

11. 前端页面设计

11.1 游客端页面

游客端建议包含以下页面：

页面	功能
游客登录页	输入姓名、身份证号、手机号登录
场馆首页	展示场馆信息、开放时间、参观须知
公告通知区域	展示已启用公告
预约时段选择页	展示可预约日期、时段、剩余名额和预约按钮

预约结果页	展示预约成功或失败原因
我的预约页	展示本人预约记录
预约凭证页	展示预约编号和二维码

11.2 管理员端页面

管理员端建议包含以下页面：

页面	功能
管理员登录页	输入用户名和密码登录后台
后台首页	展示统计概览和系统状态
场馆信息管理页	编辑场馆名称、地址、开放信息和参观须知
公告通知管理页	新增、编辑、启用、禁用公告
预约活动配置页	配置参观日期、预约时间和总名额
分时时段名额管理页	配置入场时段和时段名额
预约记录管理页	查询预约记录和游客信息
预约名单导出功能	导出 CSV 预约名单
系统状态页	查看活动状态、请求量和健康状态

11.3 前端设计原则

前端主要职责是：

- 展示数据
- 收集表单输入
- 调用后端接口
- 显示成功或失败提示

前端不能承担核心数据一致性控制。

例如：

前端可以显示“剩余名额 0，不可预约”；
但真正防止超额预约必须由后端和数据库完成。

12. 后端设计

12.1 后端包结构建议

正式后端建议采用如下结构：

```
com.example.museum
├── MuseumApplication.java
├── config
│   ├── CorsConfig.java
│   └── WebConfig.java
```

```
|—— controller
|   |—— VisitorController.java
|   |—— MuseumController.java
|   |—— ReservationController.java
|   |—— AdminController.java
|—— service
|   |—— VisitorService.java
|   |—— MuseumService.java
|   |—— NoticeService.java
|   |—— ReservationService.java
|   |—— AdminService.java
|—— repository / mapper
|—— dto
|—— entity
|—— exception
```

12.2 主要后端模块

模块	说明
用户认证模块	游客登录、管理员登录
场馆信息模块	场馆信息查询和维护
公告通知模块	公告查询和管理
预约配置模块	预约活动、预约时段、名额配置
预约核心模块	提交预约、时间校验、名额扣减、重复校验
预约记录模块	我的预约、后台预约记录、名单导出
统计模块	预约人数、剩余名额、时段统计
系统状态模块	请求量、健康状态、活动状态

12.3 统一异常处理

后端需要统一处理异常，保证前端收到固定格式响应。

常见异常包括：

- 参数格式错误
- 预约时段不存在
- 预约通道关闭
- 未到预约时间
- 预约已结束
- 名额已满
- 重复预约
- 系统内部异常

13. 测试设计初稿

13.1 功能测试

功能测试报告需要覆盖：

- 游客登录测试
- 场馆信息查看测试
- 公告查看测试
- 预约时段查询测试
- 预约成功测试
- 重复预约失败测试
- 名额已满测试
- 未到预约时间测试
- 预约已结束测试
- 管理员登录测试
- 场馆信息维护测试
- 预约活动配置测试
- 时段名额配置测试
- 预约记录查询测试
- 预约名单导出测试

13.2 性能测试

性能测试重点验证：

- 高并发下系统不崩溃
- 接口能够正常返回结果
- 不会出现超额预约
- 不会出现重复预约
- 名额数据与预约记录一致

推荐测试场景：

设置某个时段总名额为 10，同时发送 100 个预约请求。
预期结果：成功预约数 ≤ 10 ，booked_count = 成功预约记录数。

13.3 数据一致性测试

压测后需要执行数据核对：

```
SELECT slot_id, COUNT(*)
FROM reservation_record
WHERE status = 'SUCCESS'
GROUP BY slot_id;
```

再与 reservation_slot.booked_count 对比。

验收条件：

booked_count = 对应时段成功预约记录数
booked_count $\leq$ total_capacity
同一身份证同一天没有重复预约记录

14. 部署设计初稿

14.1 部署环境

部署环境要求：

- VMware 虚拟机
- Ubuntu 或 CentOS Linux
- 固定 8GB 内存
- JDK
- MySQL
- Spring Boot jar 包
- Vue 打包后的静态资源

14.2 部署步骤初稿

推荐部署步骤：

1. 前端执行 `npm.cmd run build`
2. 后端执行 `mvn -DskipTests package`
3. 将前端 dist 静态资源集成到 Spring Boot 项目
4. 在 Linux 虚拟机安装 JDK
5. 在 Linux 虚拟机安装 MySQL
6. 创建数据库并导入初始化 SQL
7. 上传后端 jar 包到虚拟机
8. 配置数据库连接参数
9. 执行 `java -jar` 启动系统
10. Windows 宿主机浏览器访问虚拟机 IP

14.3 访问方式

假设虚拟机 IP 为：

192.168.x.x

后端端口为：

8080

访问地址为：

`http://192.168.x.x:8080`

15. 风险点与解决方案

15.1 并发导致超额预约

风险：多个用户同时预约同一时段，可能导致名额超卖。

解决方案：

- 数据库条件更新
- 事务控制
- 压测验证

15.2 重复预约

风险：同一游客重复提交预约请求。

解决方案：

- 预约记录唯一约束
- 后端业务校验

15.3 前端环境复杂

风险：Vue 环境配置不熟悉，可能出现依赖或启动问题。

解决方案：

- 保持前端结构简单
- 优先实现功能闭环
- 避免一开始引入复杂状态管理

15.4 虚拟机部署失败

风险：Linux、JDK、MySQL、端口、防火墙等配置导致系统无法访问。

解决方案：

- 提前部署验证
- 记录部署步骤
- 准备部署文档和演示截图

15.5 文档材料不完整

风险：最终缺少设计文档、测试报告、性能报告或演示视频。

解决方案：

- 边开发边记录测试结果
- 提前准备文档模板
- 最终统一导出 PDF

16. 与评分标准的对应关系

评分项	分值	本设计对应内容
前后端功能完整性	30 分	游客端完整预约流程、管理员后台配置与查询功能
高并发与稳定性	25 分	并发预约设计、压测方案、接口稳定性验证
数据一致性	30 分	防超额、防重复、名额与记录一致性设计
虚拟机部署	10 分	VMware Linux、JDK、MySQL、jar 部署设计
文档规范	5 分	系统设计文档、功能测试报告、性能测试报告

17. 后续完善计划

本文档目前为系统设计初稿，后续需要继续完善以下内容：

1. 根据本文档生成正式数据库设计.md
2. 编写正式 schema.sql 初始化脚本
3. 生成后端接口设计.md, 明确请求参数和响应字段
4. 设计前端页面结构和路由方案
5. 编写功能测试报告模板
6. 编写性能测试报告模板
7. 编写 Linux 虚拟机部署文档
8. 开始正式项目代码实现

当前最建议下一步完成：

数据库设计.md

因为数据库结构确定后，后端接口、业务逻辑和前端页面才能稳定推进。